

DABAO WORKSHOP

Open, auditable silicon

NICOLAS BACCA (BTCHIP)

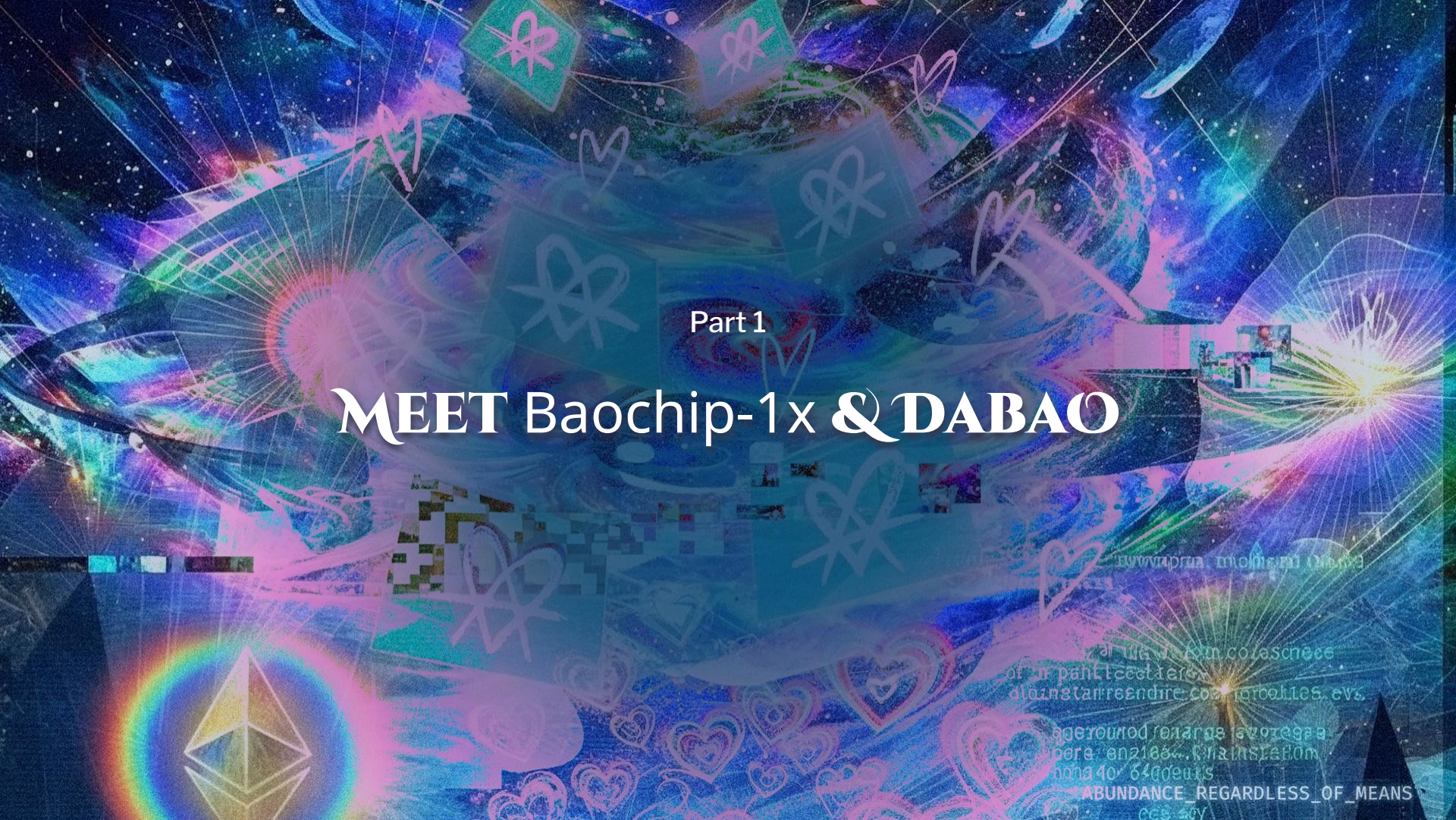
Chef, zknox

CREDITS

- Chip, board & Xous material adapted from **“Getting Started with Baochip”**
  - by bunnie (@bunnie@treehouse.systems · @bunnie.org) and baochip (@baochip.com)
  - https://baochip.com/dabao-intro/
- Getting-started steps adapted from Gerrit Niezen
  - https://gerritniezen.com/getting-started-with-the-baochip
- Boot chain & audit: xous-core sources (bao1x-boot/) and sbellem/baobit
- Slides marked with the bao logo are adapted from bunnie’s deck

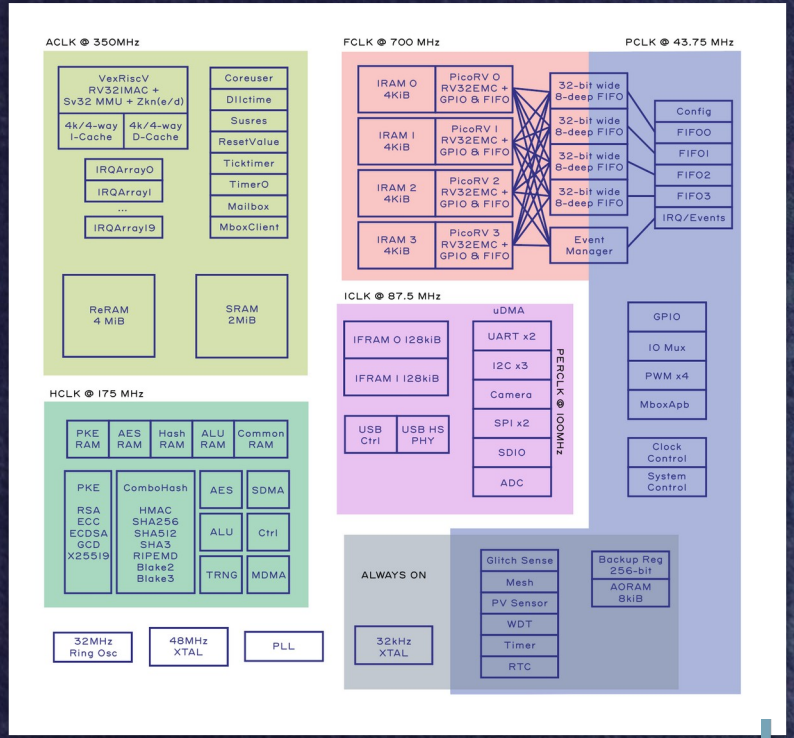
AGENDA

- **1 · Meet Baochip-1x & Dabao**: the chip, the board, IRIS inspection
- **2 · Xous**: a pure-Rust, message-passing OS (and its memory model)
- **3 · Getting started**: build, flash, console on Linux & macOS
- **4 · Boot flow**: boot0 → boot1 → loader → Xous
- **5 · Audit your chip**: read the audit report, reproduce the hashes
- **6 · Ethereum Universal Signer**: build, run & test a burner signer on Dabao


Part 1
MEET Baochip-1x & DABAO
ABUNDANCE_REGARDLESS_OF_MEANS

INTRODUCING Baochip-1x

- Five open-source CPUs
  - VexRiscv RV32IMAC + MMU @ 350 MHz
  - 4× PicoRV I/O engines (“BIO”) @ 700 MHz
- 4 MiB on-chip RRAM (non-volatile, like flash)
- 2 MiB on-chip SRAM
- Crypto accelerators, TRNG
- USB HS, camera, I2C, SPI, UART…



Adapted from “Getting Started with Baochip” by bunnie & baochip (baochip.com/dabao-intro)

Hardware words, software analogies

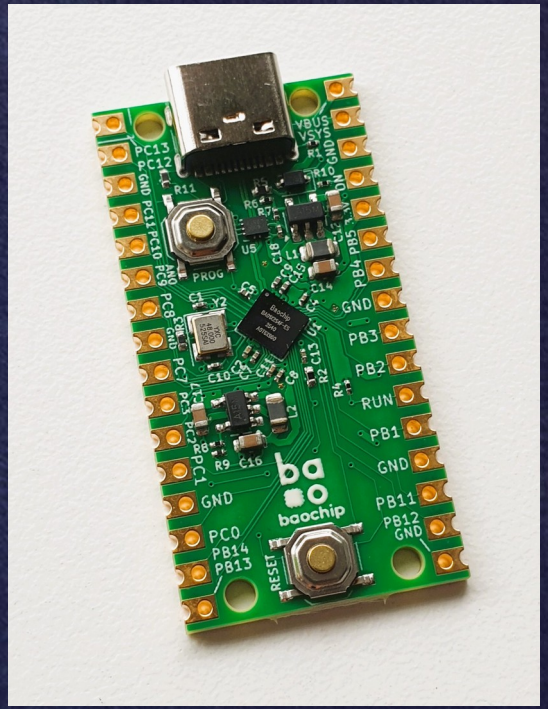
- **MCU / SoC**: a whole computer on one chip: CPU, RAM, storage, peripherals
- **RRAM**: on-chip non-volatile storage ≈ the chip's "disk", executable in place
- **SRAM**: working memory ≈ RAM
- **MMU**: hardware that maps virtual → physical pages ≈ what gives each process its own address space
- **Peripheral / MMIO**: devices (USB, UART…) controlled by reading/writing magic addresses
- **One-way counter**: a counter that can only go up, survives power-off ≈ append-only flag
- **Fuse / IFR**: configuration burned once at the factory
- **UF2**: a firmware file format you drag-and-drop onto a USB drive

How does Dabao compare?

	Dabao	Raspberry Pi Pico 2	ESP32-DevKitC	Teensy 4.1
CPU	Baochip-1x VexRiscv @ 350 MHz	RP2350 2× Cortex-M33 @ 150 MHz	ESP32 2× Xtensa LX6 @ 240 MHz	iMXRT1062 Cortex-M7 @ 600 MHz
RAM / storage	2 MiB + 256 KiB / 4 MiB RRAM	520 KB / 4 MB	520 KB / 4 MB	1 MB / 8 MB + SD
Memory protection	MMU	MPU + TrustZone	MPU-like	MPU
Security	Signed boot, key store, one-way counters, crypto accel., mesh, glitch sensors	TrustZone, signed boot, OTP keys, SHA-256, TRNG	eFuse secure boot, flash encryption	HAB secure boot, encrypted XIP
IRIS inspectable	Yes	No	No	No
Open bootloader / RTL	Yes / mostly	Yes / no	No / no	No / no
Rust-native	Yes	No	No	No
Price (approx.)	$12	$5–7	$8–12	$30–35

Adapted from “Getting Started with Baochip” by bunnie & baochip (baochip.com/dabao-intro)

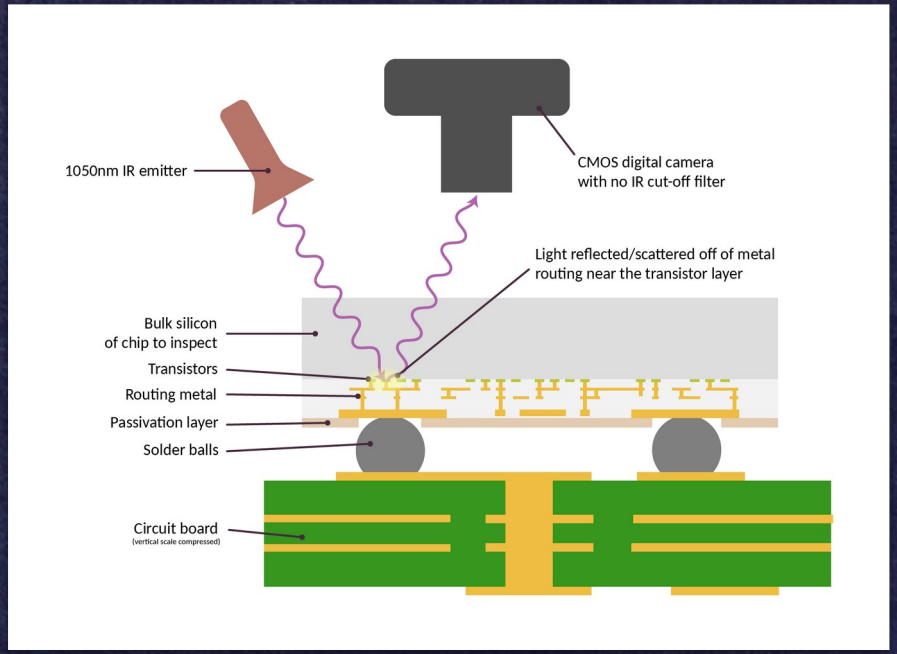
THE DABAO BOARD

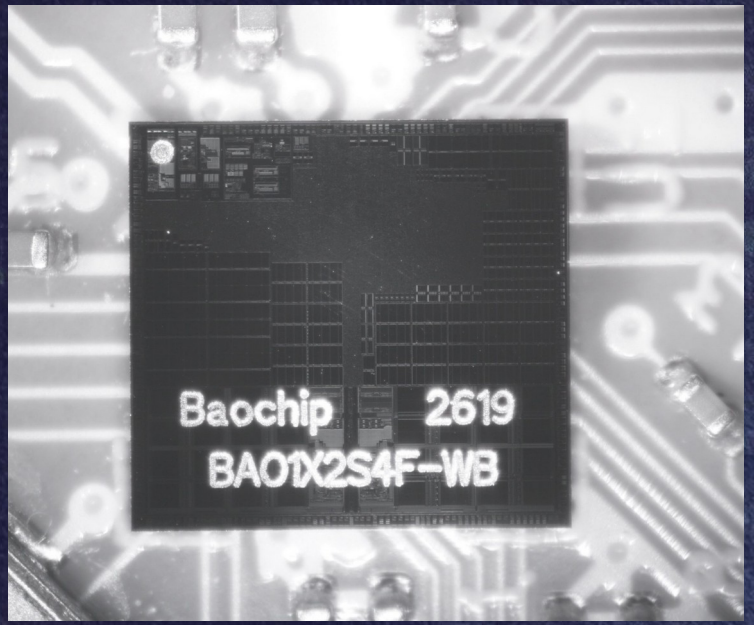


- “Simplest board”
  - 2-layer PCB, JLCPCB-compatible
  - Local voltage regulators
- Not all I/O routed to pins
  - Baochip-1x CSP package is too dense
  - A subset of routable I/O is available
- Two buttons: **PROG** and reset
- Source: https://github.com/baochip/dabao

Adapted from “Getting Started with Baochip” by bunnie & baochip (baochip.com/dabao-intro)

IRIS: INSPECT YOUR CHIP

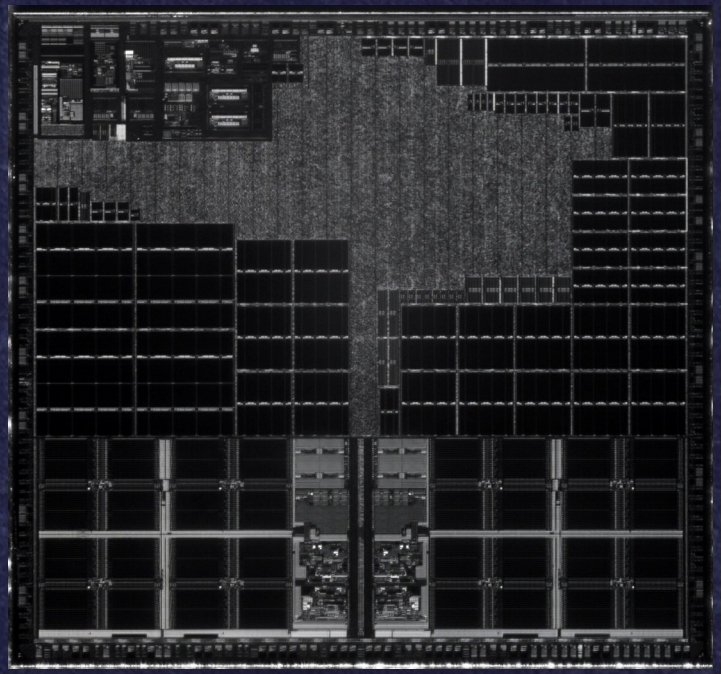




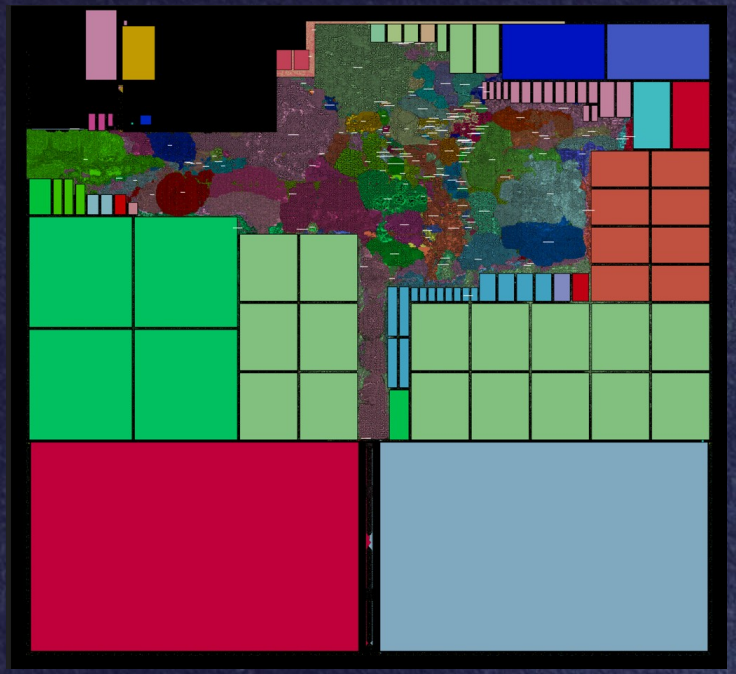
▶️ Try it live at **Devcon 8**: IRIS inspection on the **Crossbar booth (Jasmine 1)**

Adapted from “Getting Started with Baochip” by bunnie & baochip (baochip.com/dabao-intro)

IRIS MAKES FAKES HARDER



IR image of the real die



Expected functional layout

Adapted from “Getting Started with Baochip” by bunnie & baochip (baochip.com/dabao-intro)

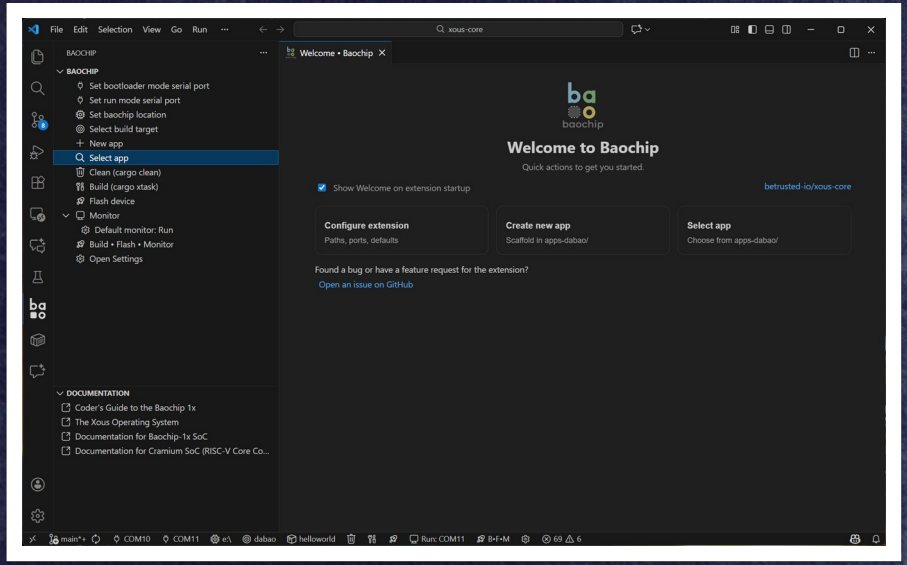


Part 2

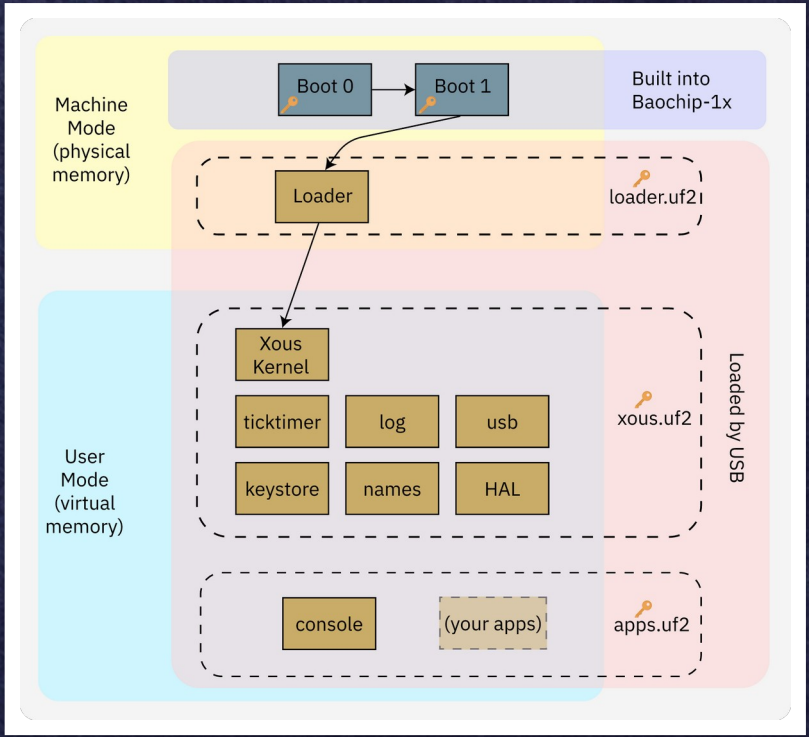
XOUS: A PURE-RUST OS

MEET XOUS

- Written from the ground up in Rust
- Source:
  - github.com/betrusted-io/xous-core
- Build for Dabao with:
  - cargo xtask dabao dabao-console
- Or use the Baochip VS Code extension
  - baochip.bao-vscode-ext



XOUS OVERVIEW

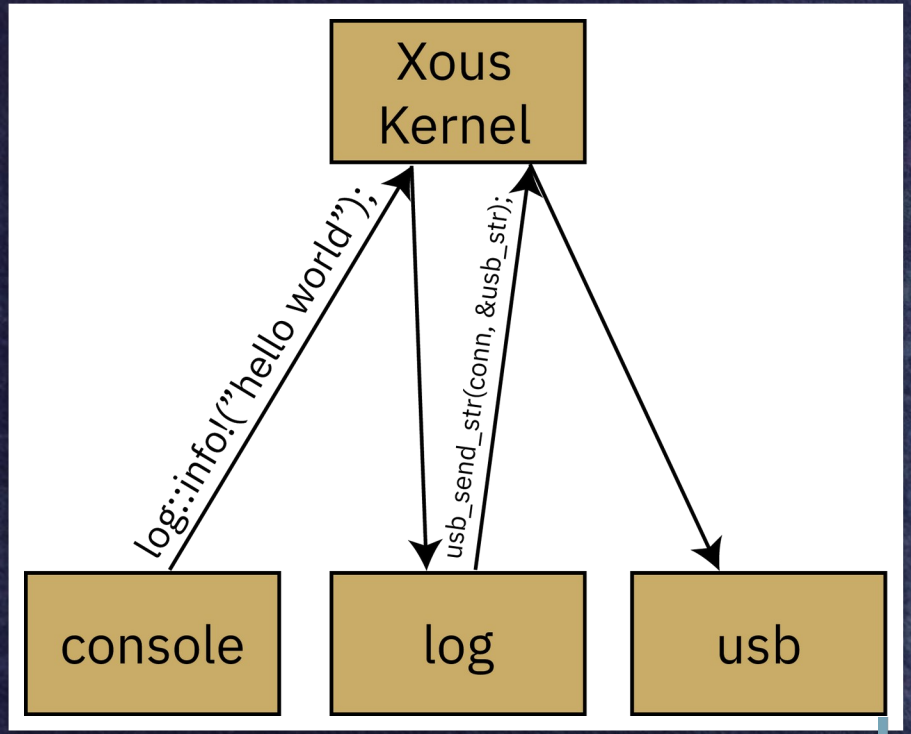


- **Built into the chip**: Boot 0 → Boot 1
- **Loaded by USB**, as three UF2 files:
  - loader.uf2: sets up the machine
  - xous.uf2: kernel + core services
  - apps.uf2: console & your apps
- Kernel and services run in **user mode with virtual memory**
- Every image is signed (key icons)

Adapted from “Getting Started with Baochip” by bunnie & baochip (baochip.com/dabao-intro)

Xous is a message-passing OS

- Every process runs in its own isolated virtual memory space
- Messages “bounce off” the kernel to reach their destination
- Example:
  - console sends a log message to log via the kernel
  - log receives it
  - and routes it to usb (and to the hardware serial port)



Adapted from “Getting Started with Baochip” by bunnie & baochip (baochip.com/dabao-intro)

Rust ownership in one minute

- Every value has **exactly one owner**
- Like a library book, you can:
  - **give it away** (move): you no longer have it
  - **lend it to read** (&T): many readers at once, nobody writes
  - **lend it to annotate** (&mut T): one writer, nobody else touches it
- The rule: **mutable XOR shared**
  - many readers, or one writer, never both
- Checked by the compiler, so no data races and no use-after-free
- C analogy:
  - move ≈ passing a malloc'd pointer and never touching it again
  - &T ≈ const T*, &mut T ≈ an exclusive T*

OWNERSHIP IN CODE

```
impl Example {
    /// The object is no longer accessible
    /// in the caller
    fn _move(self) { ... }

    /// Multiple threads can call this
    fn _borrow(&self) { ... }

    /// Only one thread may call this
    fn _mut_borrow(&mut self) { ... }
}
```

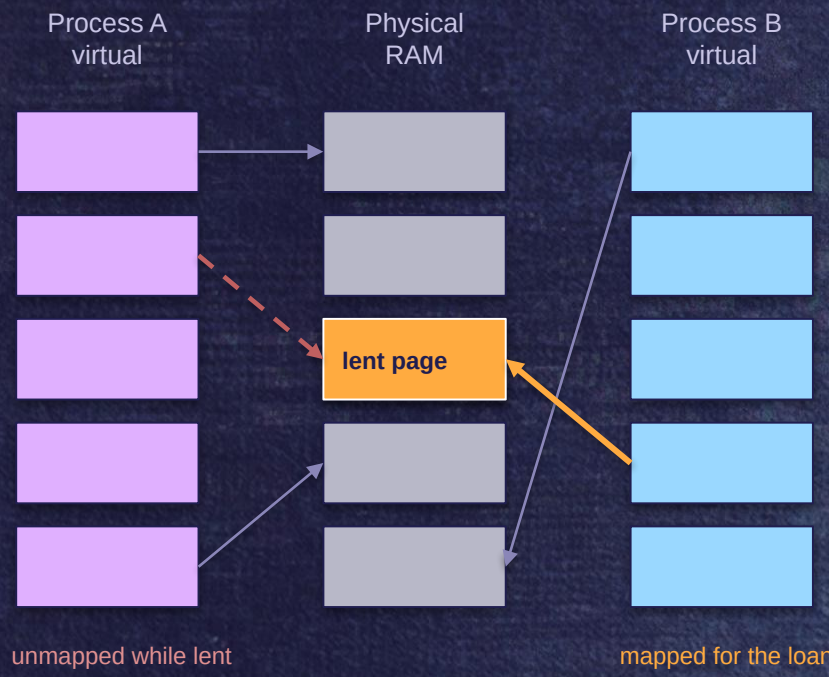
- self → give away
- &self → lend to read
- &mut self → lend to write
- Break the rule → **it doesn’t compile**

XOUS MESSAGE SEMANTICS = RUST MEMORY SEMANTICS: MUTABLE XOR SHARED

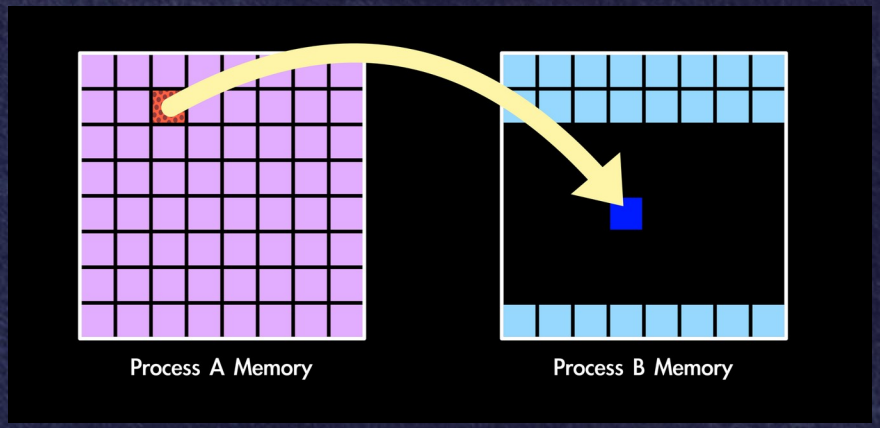
Adapted from “Getting Started with Baochip” by bunnie & baochip (baochip.com/dabao-intro)

Enforced by the MMU

- Memory is handled in **4 KiB pages**
- Each process has its own **page table**: virtual page → physical page
- No mapping, no access: A can't touch B's memory
- Sending a buffer = the kernel **re-maps the page**, no copy
- While lent, the page is **unmapped from the sender**
- So message buffers are page-aligned:
  - `#[repr(C, align(4096))]`



Lending moves memory



- The lent page is unavailable to process A until process B returns it
- Just like in Rust, the caller **blocks** during the call
- A server can re-lend the page further

Adapted from “Getting Started with Baochip” by bunnie & baochip (baochip.com/dabao-intro)

XOUS MESSAGE TYPES

Rust ownership	Memory message	Scalar message	Blocks?	Think of it as…
move	Send	Scalar	No	hand off a buffer / fire-and-forget
&borrow	Lend	BlockingScalar	Yes	sync RPC, read-only buffer
&mut borrow	LendMut	BlockingScalar	Yes	sync RPC, in/out buffer

- Scalar messages carry an opcode + 4 machine words in registers, with no memory involved
- Memory messages carry whole pages, re-mapped by the kernel

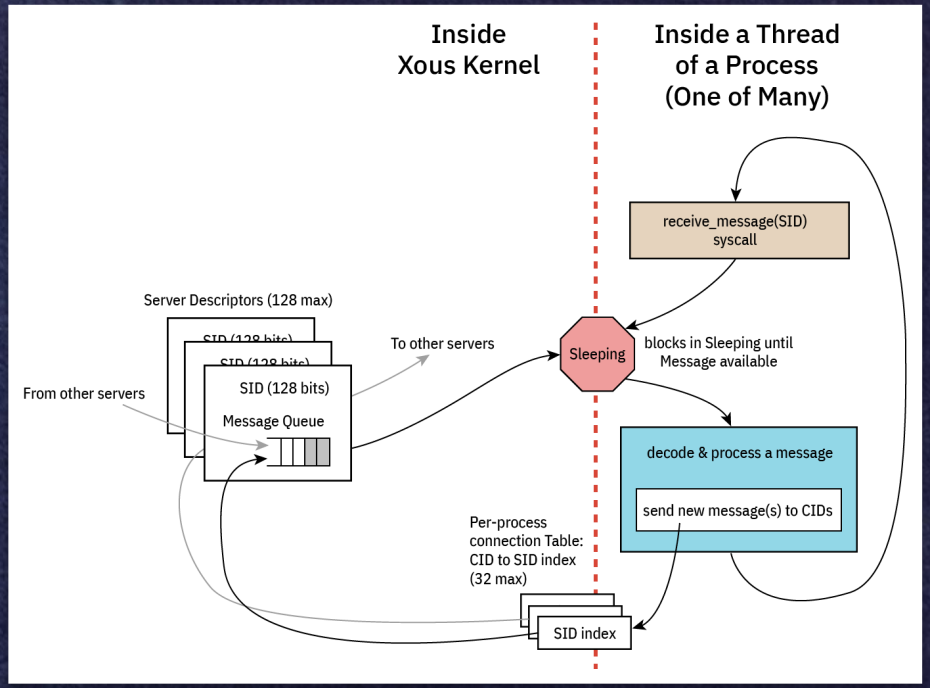
IO IS JUST MEMORY

- Peripherals are memory-mapped: registers at fixed physical addresses
- Only **one process can own** a peripheral
  - first come, first served
- Others access it through that process's message API
- That's why the **HAL server** owns most peripherals



Adapted from "Getting Started with Baochip" by bunnie & baochip (baochip.com/dabao-intro)

SERVERS & CONNECTIONS




```
// server
let srv = create_server(b"0123456789abcdef");
while let Some(mut msg) = receive_message(srv) {
   // handle message
}
```

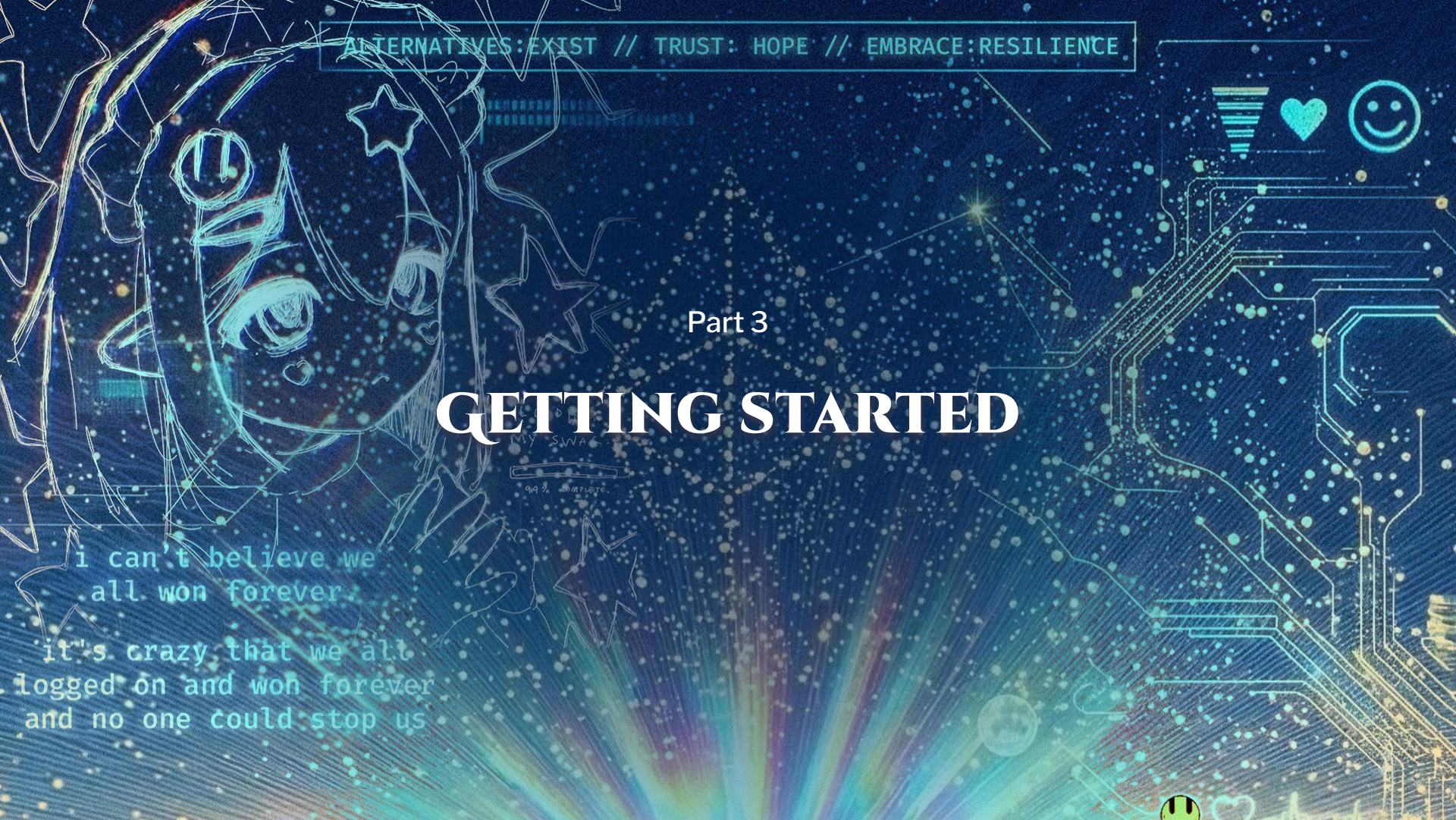
```
// client
let conn = connect(b"0123456789abcdef");
scalar_message(conn, 1, 2, 3, 4, 5);
lend_message(conn, 1, &slice, 4, 5);

// e.g. sleep = BlockingScalar to ticktimer
blocking_scalar(ticktimer, SLEEP_MS,
            dur.as_millis(), 0, 0, 0);
```

Adapted from “Getting Started with Baochip” by bunnie & baochip (baochip.com/dabao-intro)

LIBSTD: RUST'S STANDARD LIBRARY ON XOUS

- Upstream **Tier-3 Rust target**: riscv32imac-unknown-xous-elf
- Only stable Rust features
- No C compiler or linker needed
- Rust semantics enforced by the **hardware MMU**
- cargo xtask offers to install the target toolchain on first build



Part 3

GETTING STARTED

⚠️ Want to audit your board? Do it first

- Follow the next slides and run audit (part 5) **before flashing anything**
- Anything you build yourself is signed with the **public developer key**
- The first time boot1 sees a dev-signed image, it **erases the chip's factory secrets** and switches to **developer mode**
  - one-way counter: **no undo**, and the audit then reports a developer device
- Don't care about auditing? Fine: Dabao is a dev board, flash away (part 6)

What you need

- A Dabao + USB-C data cable
- A serial terminal: **screen**
- For part 6 (building an app):
  - **git**
  - **Rust** via rustup; already installed? rustup update

LINUX: CONNECT

```
# plug in the Dabao: no app loaded, it waits in boot1

# 1. find the serial port
sudo dmesg | grep tty         # e.g. cdc_acm ...: ttyACM0: USB ACM device
sudo usermod -aG dialout $USER # once, then log out / in

# 2. connect
sudo apt install screen       # or your distro's package manager
screen /dev/ttyACM0 1000000    # quit: ctrl-a \
```

- Dabaos ship **with no application**: you land in **boot1**, audit works right away
- Leftover screen sessions hold the port: ps aux | grep -i screen

MACOS: CONNECT

```
# 0. toolchain (for building in part 6)
xcode-select --install                  # git + linker
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# plug in the Dabao: no app loaded, it waits in boot1
# 1. find the port: use cu.*, not tty.*; no dialout group needed
ls /dev/cu.usbmodem*
screen /dev/cu.usbmodemXXXX 1000000       # quit: ctrl-a \
```

- screen is preinstalled on macOS (brew install screen if missing)
- Same as Linux from here: Enter, then audit (part 5)

IN THE boot1 CONSOLE

- Press Enter, then type any unknown word (e.g. help) → list of commands
- **Read-only**, safe to try:
  - audit · bootwait check · boardtype · altboot · idmode · usb_speed · echo
- localecho on | off if typing looks doubled or invisible

- **Change one-way counters**: only when you mean it
  - bootwait enable | disable · boardtype <type> · altboot toggle · usb_speed full | high
- **Irreversible**: don't
  - lockdown · paranoid enable · require-pq · self_destruct
- USB serial disconnects at each stage change: re-open screen



Part 4

BOOT FLOW: boot0 → boot1 → XOUS

THE CHAIN OF TRUST

boot0 ◦ 0x60000000

Burned at the factory, **immutable**
Root of trust: 4 ed25519 public keys
Checks itself and boot1

boot1 ◦ 0x60020000

Updatable, signed by Baochip
USB drive + serial console
Checks the next stage, then jumps

LOADER ◦ 0x60060000

Signed next stage (or baremetal)
Starts the Xous kernel
Kernel loads services & apps

boot0: ROOT OF TRUST

- Written at chip probe, then **sealed** (JTAG fused out)
- Runs at a conservative 200 MHz; checks code in ~70 ms
- Verifies **itself** and **boot1** against up to **4 ed25519 public keys**
  - slots: bao1 · bao2 · beta · developer
- Key slot skipped if all-zero or **revoked** (one-way counter)
- Developer key matched → erase secrets, enter **developer mode**
- No valid key → zeroize, print XXXX on serial, hang
- boot1 invalid → fall back to a fail-safe image in the loader slot

boot1: THE UPDATABLE LOADER

- Board-specific: Dabao, Baosec, OEM variants
- Default: validate the next stage and run it
- **Boot-wait** (button held, or bootwait counter) → stays in boot1:
  - USB mass-storage: drop .uf2 files
  - USB-serial console (+ physical UART PB13/PB14 @ 1 Mbaud)
- Writes partitions: loader/baremetal, kernel, apps
- Console commands include:
  - audit bootwait boardtype lockdown paranoid reset…
- Updating boot1 itself goes through a signed **updater** image (v0.10.2+)

KEYS, COUNTERS & DEVELOPER MODE

- Every stage carries its own copy of the **public keys**; the chip stores **receipts** of what it last validated
- **One-way counters** record irreversible state: developer mode, key revocations, anti-rollback, boot-wait
- Developer key anywhere in the chain → **secrets erased** + DEVELOPER_MODE counter bumped
- lockdown in boot1 revokes the developer key (Baosec boards ship this way)
- Goal: anyone can run their own code, but **not with the factory's secrets**


Part 5
AUDIT YOUR CHIP
ABUNDANCE_REGARDLESS_OF_MEANS

RUNNING THE AUDIT

1. Enter **boot1**: plug in Dabao (or hold **PROG** while plugging in)
2. BAOCHIP drive appears; open the serial port with screen
3. Press Enter, type audit
4. Save the output

Runs automatically on the first 3 boots (auto-audit limit)

```
$ screen /dev/cu.usbmodemXXXX 1000000
audit
Board type reads as: Dabao
Boot partition is: Ok(PrimaryPartition)
Semver is: v0.10.0-61-g5397e1b48
Description is: bao2-0
Stepping is: A0
Device serializer: 541acb9a-9396c3ca-...
Public serial number: KAC0Q6
UUID: 0e4c8563-127db16d-...
Paranoid mode: 0/0
Possible attack attempts: 0
Revocations:
...
```

READING THE AUDIT (1/3): IDENTITY

- Board type: Dabao (one-way counter)
- Boot partition: PrimaryPartition = normal boot1, not the fail-safe
- Semver: git describe of the boot1 source: v0.10.0 + 61 commits = **commit 5397e1b48**
- Stepping: **A0** = pre-production silicon
- Device serializer, Public serial number, UUID: unique per chip
- Paranoid mode 0/0: extra hardening off
- Possible attack attempts 0: glitch-sensor counter (can be non-zero, sensors are on a hair trigger)

READING THE AUDIT (2/3): SIGNATURES

- Revocations: all **enabled** = no key revoked
- Boot0: key 0/0 (bao1) -> 60000000
- Boot1: key 0/0 (bao1) -> 60020000
  - validated by Baochip release key **bao1**
- Next stage: key 2/2 (beta) -> 60060000
  - factory loader signed with the **beta** key

- After flashing your own build: **key 3 (dev)** + == IN DEVELOPER MODE ==
- Erase proof: uninit or access denied: no dev-mode erase has happened
- auto-audit limit: 3: automatic audits used up

READING THE AUDIT (3/3): FINAL CHECKS

- In-system keys have been generated
- CM7 & debug confirmed fused off
  - IFR bits: the unused Cortex-M7 & debug port are disabled
- Collateral erased
- Boot1 receipts OK: boot1 keys match what the chip last recorded
- Boot1 anti-rollback OK

- **What you must NOT see:**
  - == IN DEVELOPER MODE ==
  - == BOOT1 FAILED PUBKEY CHECK ==
  - Factory configuration error...
  - ** System did not meet minimum requirements for security **
- The sample is clean ✔️

DO THE HASHES MATCH THE RELEASE?

boot0 code only: 563802e7f10fd0ca0a1c...a1be4c2b4dee2
boot0 baobit toolchain: 441559d7e1984623ec0a52f60f90240c740b6c41
boot1 code only: e42b5aba61d98990ca24...a90f2dc4551a55
boot1 baobit toolchain: 441559d7e1984623ec0a52f60f90240c740b6c41
(+ "boot0/boot1 partition" hashes = whole partition incl. free space)

- ✔️ All six values match the **v0.10.0 references** in xous-core/bao1x-boot/README.md
- **code only** = signed code without signature block: the reproducible part
- **baobit toolchain** = commit of the reproducible-build toolchain that built it

REPRODUCE THE HASHES YOURSELF

```
# needs GNU Guix (Linux; on macOS use a Linux VM)
BAOBIT_COMMIT_SHORT=441559d7      # from "baobit toolchain" in your audit
curl --proto '=https' --tlsv1.2 -sSfLo baobit.scm \
  "https://raw.githubusercontent.com/sbellem/baobit/refs/heads/main/channels/baobit.${BAOBIT_COMMIT_SHORT}.scm"

guix time-machine --channels=baobit.scm -- build bao1x-boot0 --root=boot0
guix time-machine --channels=baobit.scm -- build bao1x-boot1 --root=boot1

sha512sum boot0/bao1x-boot0-presign.img   # == "boot0 code only"
sha512sum boot1/bao1x-boot1-presign.img   # == "boot1 code only"
```

- First build compiles the Rust toolchain: slow and disk-hungry, **do it at home**
- Match = the code on your chip was built from the published source

What the audit proves (or not)

- ✔ boot0/boot1 on the chip = published, reproducible source
- ✔ no key revoked, not in developer mode, debug fused off
- ✔ three-way check: device · your rebuild · official release

- ✘ boot1 reports on itself: a malicious boot1 could lie
- ✘ relies on an **honest chip-probe** step at the factory
- ✘ **A0 stepping**: boot0 write-protection may not be enforced
- → combine with **IRIS** to check the silicon itself (Crossbar booth, Jasmine 1)

YOUR boot1 MAY PRINT MORE

- Newer boot1 versions extend the report (current boot1/src/audit.rs):
  - First-try boot partition and Program counter is in
  - PQ required: post-quantum signature policy
  - arb N (anti-rollback) and pq tag on each stage
  - Swap key is set / not set
- Compare hashes against the references **for your version** (bao1x-boot/README.md)

Part 6

ETHEREUM UNIVERSAL SIGNER ON DABAO



COMPONENTS OF A HARDWARE WALLET

Seed management

Always needed, at least as a generator for other secrets. Stateful or stateless.

Cryptography

Curves, hashes, BIP-32/39 derivation, entropy

Genuineness

Prove the device is what it claims to be

Ethereum signing

Decode, display, sign: transactions, EIP-712, EIP-191

Application management

Load, isolate, update applications

UX

Screens and inputs: what the user actually verifies

Transport

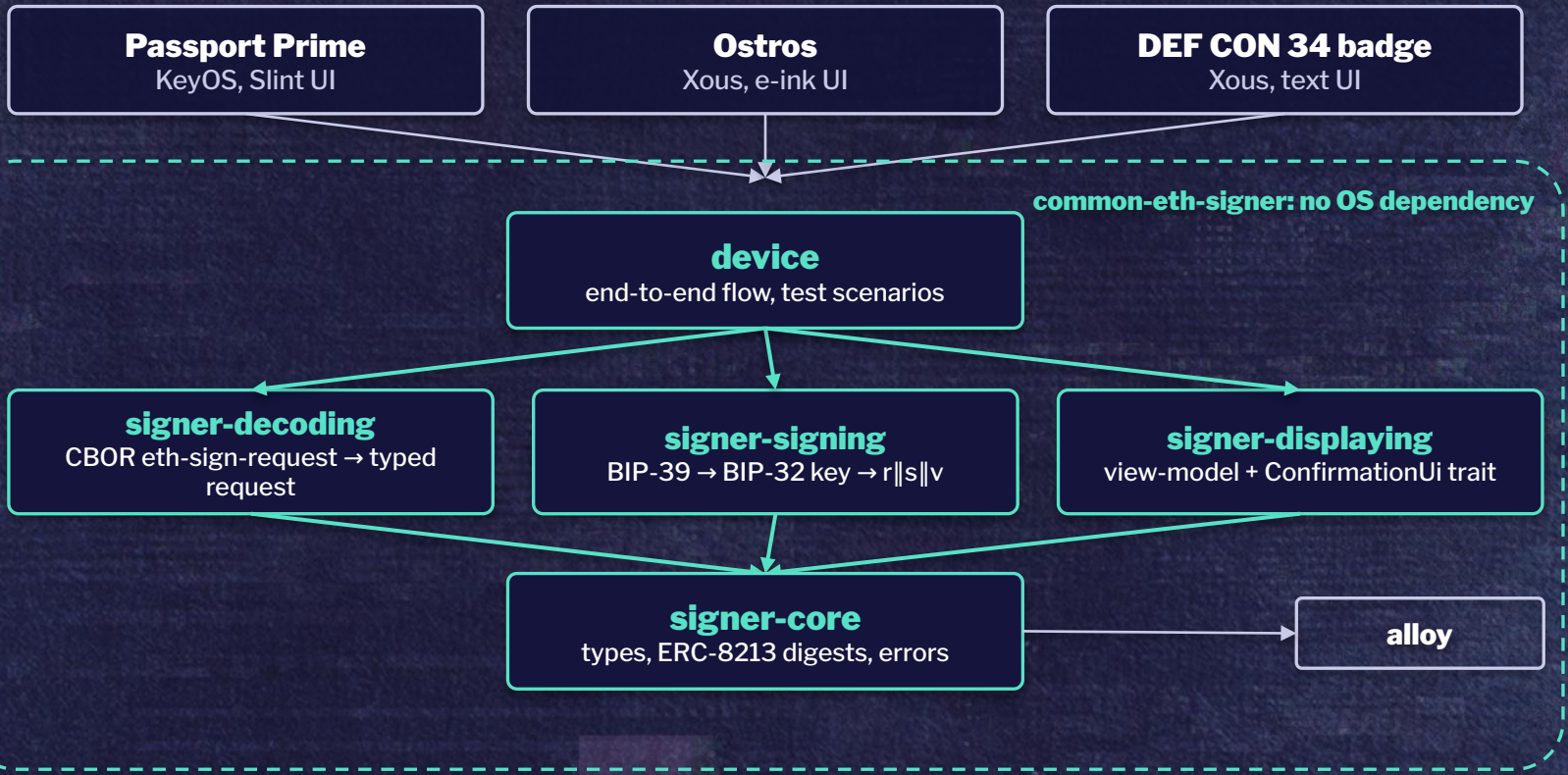
QR (ERC-4527), USB, BLE

Each component can be tackled separately, by different teams or agents.

OPINIONATED ETHEREUM SIGNING

- **Rust**: more guarantees at compile time, ideal for agents, built on alloy
- **Airgap-ready**: ERC-4527 QR requests and signatures
- **Minimalistic**, while promoting modern standards: pick only the modules you need
- **CROPS in mind**: verifiable signing (ERC-8213) before clear signing (ERC-7730)

INSIDE COMMON-ETH-SIGNER



EXISTING IMPLEMENTATIONS

Passport Prime
Foundation

- Commercial
- KeyOS multi-app OS
- Slint UI
- Camera (QR)
- Later: USB, BLE

Ostros
Xous · ostros profile

- Commercial
- Xous
- e-ink touch UI
- USB HID
- Later: BLE

DEF CON 34 badge
Xous · baosec-lite

- Text UI, jog dial
- Custom-focus camera (QR)

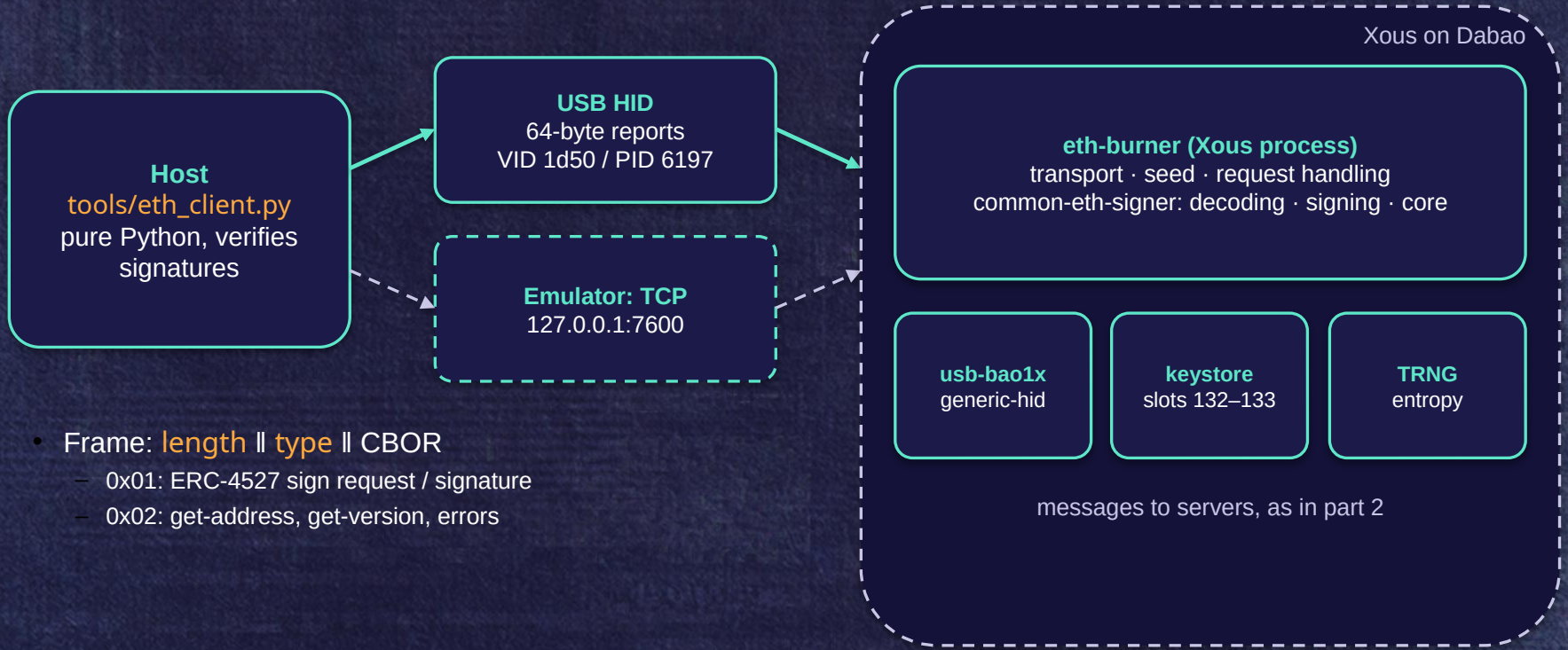
Dabao
Xous · dabao profile

- No UI
- Bare signer for developers

THE DABAO BURNER

- dabao-app-eth-burner: the Universal Signer for a **bare Dabao**: no screen, no buttons
- First boot: random seed from the chip **TRNG** (24-word equivalent)
  - stored in keystore app slots 132–133, never shown or exported
- Talks to the host over **USB HID** (emulator: TCP socket)
- Answers at once, **without review**:
  - get-version · get-address · transactions (legacy, EIP-1559, EIP-7702) · EIP-712 · EIP-191
- ⚠️ Anyone who can reach the USB port can sign: **test networks & small amounts only**

How it fits together



- Frame: length ‖ type ‖ CBOR
  - 0x01: ERC-4527 sign request / signature
  - 0x02: get-address, get-version, errors

CLONE

```
mkdir dabao-workshop && cd dabao-workshop
git clone -b devcon8-workshop https://github.com/btchip/xous-core xous-core
git clone -b devcon8-workshop https://github.com/btchip/ethereum-app ethereum-app

# version tags from upstream (image signing needs `git describe`)
git -C xous-core fetch https://github.com/betrusted-io/xous-core 'refs/tags/*:refs/tags/*'
cd xous-core && cargo xtask install-toolkit && cd ..     # Xous toolchain, once
```

```
dabao-workshop/
├── xous-core/
└── ethereum-app/
    └── dabao-app-eth-burner/
```

- Must be **side by side**: the app uses ../../xous-core crates
- Branch **devcon8-workshop** for both
- Full clone of xous-core, **no** --depth 1

BUILD

```
cd ethereum-app/dabao-app-eth-burner
./build.sh emu      # host binary   -> target/release/eth-burner
./build.sh device   # Xous binary   -> target/riscv32imac-unknown-xous-elf/release/eth-burner
                    # (./build.sh = both)

# device image: app path first, then the flags
cd ../../xous-core
APP=../ethereum-app/dabao-app-eth-burner/target/riscv32imac-unknown-xous-elf/release/eth-burner
cargo xtask dabao $APP --feature usb-bao1x/generic-hid
```

- generic-hid: one vendor HID interface replaces keyboard + FIDO + **serial console**
- Output: the usual loader.uf2 xous.uf2 apps.uf2, with eth-burner inside apps.uf2

RUN IN THE EMULATOR FIRST

```
# terminal 1: from xous-core/
cargo xtask dabao-emu ../ethereum-app/dabao-app-eth-burner/target/release/eth-burner
INFO:eth_burner: new burner seed created and stored
INFO:eth_burner: burner address m/44'/60'/0'/0/0: 0x06A77bCcC40eda354e615B0934430d40BE2d986d
INFO:eth_burner::transport: emulator transport: listening on 127.0.0.1:7600
```

```
# terminal 2: from ethereum-app/dabao-app-eth-burner/
tools/eth_client.py get-version      # version 0.1.0
tools/eth_client.py get-address      # address 0x06A77bCcC40eda354e615B0934430d40BE2d986d
```

- Same app, TCP 127.0.0.1:7600 instead of USB, a host file instead of the keystore

RUN ON THE DABAO

```
# 1. plug in the Dabao -> USB drive BAOCHIP (later: hold PROG while plugging in)
cd xous-core/target/riscv32imac-unknown-xous-elf/release
cp loader.uf2 xous.uf2 apps.uf2 /media/$USER/BAOCHIP/      # Linux
cp -X loader.uf2 xous.uf2 apps.uf2 /Volumes/BAOCHIP/       # macOS
# 2. press PROG: Xous starts with eth-burner

# 3. from ethereum-app/dabao-app-eth-burner/
source ~/dabao-venv/bin/activate        # hidapi venv (workshop_prep.md)
tools/eth_client.py --hid get-address    # your burner address
```

```
# Linux only: /etc/udev/rules.d/70-dabao-hid.rules
SUBSYSTEM=="usb", ATTRS{idVendor}=="1d50", ATTRS{idProduct}=="6197", TAG+="uaccess"
KERNEL=="hidraw*", ATTRS{idVendor}=="1d50", ATTRS{idProduct}=="6197", TAG+="uaccess"
```

- ⚠️ Dev-signed image → **developer mode**: run audit first
- No USB console any more: boot log only on the UART (PB13/PB14, 1 Mbaud)

TEST: SIGN AND VERIFY

```
A=0x...   # your burner address
tools/eth_client.py sign-tx --expect $A                # EIP-1559 (--legacy: EIP-155)
tools/eth_client.py sign-typed-data mail --expect $A     # EIP-712 (group | FILE.json)
tools/eth_client.py sign-message 'Hello' --expect $A     # EIP-191 (siwe | 0xHEX)
#   signer     0x06A77bCcC40eda354e615B0934430d40BE2d986d    <- recovered, == $A

# negative test: someone else's address
tools/eth_client.py sign-tx --address 0xA27DF20E6579aC472481F0Ea918165d24bFb713b
#   error 2 (invalid-request): address mismatch: not this device's key
```

- The client recovers the signer from the signature: **you don’t trust the device’s word**
- Add --hid on the board; nothing extra for the emulator

GOING FURTHER

- Hack the app: src/main.rs (seed & signing), src/transport.rs, protocol.cddl
- Re-flash just apps.uf2 when only the app changed
- Start the app at power-up, no PROG press: bootwait disable in the **boot1** console
  - back to boot1 any time: hold PROG while plugging in
- Want a Xous shell instead? cargo xtask dabao dabao-console, loaded the same way
- Reuse common-eth-signer on your own device: it has **no OS dependency**
- Hardware checklist to help validate: xous-core/apps-dabao/HARDWARE-TESTS.md
- Not audited yet: **assets you can afford to lose**

RESOURCES

- Getting Started with Baochip (bunnie): baochip.com/dabao-intro
- Xous source & docs: github.com/betrusted-io/xous-core
  - README-baochip.md · bao1x-boot/BOOTCHAIN.md · README-consoles.md
- Dabao hardware: github.com/baochip/dabao
- Getting started (Gerrit Niezen): gerritniezen.com/getting-started-with-the-baochip
- Reproducible builds: github.com/sbellem/baobit (VERIFY.md)
- Universal Signer: github.com/btchip/ethereum-app · github.com/btchip/xous-core (branch devcon8-workshop)
- IRIS live: **Devcon 8, Crossbar booth (Jasmine 1)**
- Dabao Hackathon: crossbar-inc.com/pages/dabao-hackathon

CROSSBAR DABAO HACKATHON

- **“Show us something cool on an open chip.”**
- Open brief: any stack, any idea
- Up to **$5,000** for the most impressive builds
- Boards: CrossBar Impact Space, Jio World Centre
- Submit one link (source published) by **Dec 15, 9 pm ET**
- Judged on: works · new · open · uses the silicon

crossbar-inc.com/pages/dabao-hackathon



THANK YOU!

NICOLAS BACCA (BTCHIP)

Chef, zknox